Machine Failure Prediction Using the AI4I 2020 Predictive Maintenance Dataset

1. Abstract

This project builds a supervised binary classifier for machine failure prediction using the AI4I 2020 Predictive Maintenance Dataset from the UCI Machine Learning Repository. The model uses product type and five numeric operating measurements as inputs. Diagnostic failure mode columns are excluded from model training because they leak target information. Five classifiers were compared using a stratified 70/15/15 train/validation/test split. The selected model was HistGradientBoostingClassifier, chosen by validation F1-score. On the held-out test set, it reached F1-score 0.7556, recall 0.6667, F2-score 0.6996, and ROC-AUC 0.9750.

2. Introduction and Motivation

Predictive maintenance uses machine data to estimate whether equipment is likely to fail. In real settings, a missed failure can cause downtime, repair costs, safety issues, and production delays. This makes recall and F2-score important secondary metrics, while F1-score remains the main balanced metric for model selection.

The AI4I dataset was chosen because it is directly related to machine condition monitoring, has a clear binary failure label, and is small enough for a course project where the full workflow should be easy to rerun. The goal of this project is to keep the workflow simple and reproducible: load a known dataset, prevent leakage, compare standard classifiers, save a final model, and provide a local demo.

3. Related Work or Background

Predictive maintenance is commonly framed as a classification, regression, anomaly detection, or survival analysis problem. This project uses supervised classification because the AI4I dataset includes a ready-made binary target. Tree ensembles and gradient boosting are often strong baselines for tabular machine learning because they can model nonlinear interactions between operating conditions.

4. Dataset Description

The dataset is the AI4I 2020 Predictive Maintenance Dataset from the UCI Machine Learning Repository. The local raw file is `data/raw/ai4i2020.csv`. It contains 10,000 synthetic observations. The input data includes one categorical feature, `type`, and numeric operating measurements for temperature, rotational speed, torque, and tool wear.

Target:

- `machine_failure = 0`: no machine failure
- `machine_failure = 1`: machine failure

Approved model inputs:

- `type`
- `air_temperature_k`
- `process_temperature_k`
- `rotational_speed_rpm`
- `torque_nm`
- `tool_wear_min`

Dropped ID columns:

- `UDI`
- `Product ID`

Diagnostic failure mode columns excluded from model features:

- `TWF`
- `HDF`
- `PWF`
- `OSF`
- `RNF`

5. Preprocessing and Exploratory Data Analysis

Preprocessing loads the raw CSV, cleans column names into snake_case, drops ID columns, and writes `data/processed/ai4i_processed.csv`. The processed file keeps diagnostic failure mode columns for explanation only, not for modeling.

Generated EDA outputs include:

- `results/plots/class_balance.png`
- `results/missing_values_summary.csv`
- `results/plots/feature_distributions.png`
- `results/plots/correlation_heatmap.png`
- `results/plots/target_vs_features.png`
- `results/plots/failure_mode_counts.png`

The missing values summary shows zero missing values in all processed columns.

Feature engineering and representation are intentionally simple. The project performs column-name cleaning, feature selection, one-hot encoding for `type`, and numerical standardization inside the modeling pipeline. No additional synthetic features are created. Feature selection is important because it removes IDs and excludes the diagnostic failure mode columns that would leak target information.

Feature distribution plots and target-vs-feature boxplots provide visual outlier review. No rows are removed as outliers because the values represent operating conditions in the synthetic dataset and may be meaningful for failure prediction.

6. Methodology and Models

The project trains exactly five models:

Model	Notes
Logistic Regression	Baseline with `class_weight="balanced"`
Decision Tree	Tuned with class weighting
Random Forest	Tuned with class weighting
Extra Trees	Fixed baseline ensemble with class weighting
HistGradientBoostingClassifier	Tuned gradient boosting model

The categorical `type` feature is one-hot encoded. Numeric features are standardized in the common preprocessing pipeline.

7. Validation and Hyperparameter Tuning Strategy

The split is stratified using `random_state=42`:

- Train: 70%
- Validation: 15%
- Test: 15%

Decision Tree, Random Forest, and HistGradientBoostingClassifier are tuned with small grids on the validation set. The validation set is used for model selection. The test set is reserved for final evaluation.

Tuning grids:

- Decision Tree: `criterion`, `max_depth`, `min_samples_leaf`
- Random Forest: `n_estimators`, `max_depth`, `min_samples_leaf`

- HistGradientBoostingClassifier: `learning_rate`, `max_iter`, `max_leaf_nodes`

8. Experimental Setup

The authoritative scripts are:

- `python -m src.preprocessing`
- `python -m src.eda`
- `python -m src.train`
- `python -m src.evaluate`
- `python demo/demo.py`

The final model is saved to `models/final_model.joblib`. Validation metrics are saved to `results/metrics_table.csv`, and final test metrics are saved to `results/test_metrics.csv`.

9. Results and Model Comparison

Validation results:

Rank	Model	F1-score	Recall	F2-score	ROC-AUC
1	HistGradientBoostingClassifier	0.7742	0.7059	0.7317	0.9872
2	Decision Tree	0.7059	0.7059	0.7059	0.8478
3	Random Forest	0.6105	0.5686	0.5847	0.9766
4	Extra Trees	0.3939	0.2549	0.2968	0.9434
5	Logistic Regression	0.2298	0.7255	0.3895	0.8750

Final held-out test results for HistGradientBoostingClassifier:

Metric	Value
Accuracy	0.9853
Precision	0.8718
Recall	0.6667

Metric	Value
F1-score	0.7556
F2-score	0.6996
ROC-AUC	0.9750

Test confusion matrix counts:

- True negatives: 1444
- False positives: 5
- False negatives: 17
- True positives: 34

10. Error Analysis and Qualitative Discussion

The selected model had high precision and strong ROC-AUC, but recall was lower than precision. This means the model was conservative when predicting failures: false alarms were rare, but some true failures were missed. In a maintenance setting, false negatives can be expensive, so future work should examine probability threshold tuning or cost-sensitive learning.

On the held-out test set, the model caught 34 failure cases and missed 17 failure cases. It also produced 5 false alarms. Accuracy is high, but these missed failures are important because recall matters when the positive class represents a costly machine failure.

The Logistic Regression baseline had much higher recall than its F1-score suggests, but it produced many more false positives. This tradeoff is useful for discussion because different maintenance settings may prefer higher recall even at the cost of more inspections.

11. Demo or Usage Demonstration Description

The demo script `demo/demo.py` loads `models/final_model.joblib` and sample rows from `data/processed/ai4i_processed.csv`. It prints each sample's true class, predicted class, and predicted probability of machine failure. The demo is local and simple; it is not deployment software or a production maintenance system.

12. Limitations and Future Work

- The AI4I dataset is synthetic, so real-world performance is unknown.
- The positive class is rare, making recall-sensitive evaluation important.
- The project does not optimize the decision threshold after model selection.

- The workflow does not include live monitoring, streaming data, retraining, or deployment.
- Future work could add threshold tuning, calibration, cost-sensitive evaluation, and robustness checks on real maintenance data.

13. Conclusion

Overall, this project shows a clear binary classification workflow for machine failure prediction. It avoids leakage by excluding failure mode columns from model features, uses validation-based model selection, saves a final model, and evaluates once on the held-out test set. HistGradientBoostingClassifier achieved the best validation F1-score and produced a test F1-score of 0.7556.

14. Team Contributions

- Niraj Patel — coordinated the repository, integrated the main workflow, organized the dataset pipeline, implemented the preprocessing/training/evaluation scripts, prepared the demo, and assembled the documentation and reproducibility checks.
- Samuel Lavallée — supported dataset review and helped check the feature/target definitions.
- Thinoushan Senathirajah — supported EDA review and helped check plots and class balance observations.
- Omar Shrit — supported model result review and helped check validation/test metric interpretation.
- Arnav Singh — supported final report review, demo instructions, and submission checklist review.

15. References

1. 1. UCI Machine Learning Repository. AI4I 2020 Predictive Maintenance Dataset. https://archive.ics.uci.edu/dataset/601/ai4i+2020+predictive+maintenance+dataset
2. 2. scikit-learn developers. scikit-learn: Machine Learning in Python. https://scikit-learn.org/
3. 3. pandas development team. pandas documentation. https://pandas.pydata.org/

Academic integrity and external tools acknowledgment: this project uses standard Python libraries including pandas, scikit-learn, matplotlib, seaborn, joblib, and nbformat. AI-assisted coding and documentation support was used to help scaffold and review the repository; the team remains responsible for understanding, verifying, and disclosing this use according to the course policy.